

Praktikum 7: Suche in Graphen

Aufgabe 1: Breitensuche auf einer Höhenkarte

Sie erhalten eine Höhenkarte in Form einer ASCII-Darstellung. Jedes Zeichen entspricht einem Feld; der Buchstabe gibt die relative Höhe an (a = niedrigste, z = höchste Höhe). Das Zeichen S markiert Ihren Startpunkt (Höhe wie a), das Zeichen E das Ziel (Höhe wie z).

Von einem Feld aus können Sie in einem Schritt auf ein horizontal oder vertikal benachbartes Feld wechseln – jedoch nur dann, wenn das Zielfeld höchstens eine Höhenstufe höher liegt als das aktuelle. Ein Wechsel bergab oder auf gleicher Höhe ist immer erlaubt.

Das Beispielgelände:

```
Sabqponm  
abcryxxl  
accszExk  
acctuvwj  
abdefghi
```

Die Aufgabenstellung entspricht „Advent of Code“ von 2022, Tag 12.

(<https://adventofcode.com/2022/day/12>)

- Modellierung:** Modellieren Sie die Karte als gerichteten Graphen: Jedes Feld (Zeile, Spalte) ist ein Knoten; eine gerichtete Kante existiert von Feld A nach Feld B, wenn B ein horizontaler oder vertikaler Nachbar von A ist und die Bewegungsregel eingehalten wird. Welche Darstellung (Adjazenzliste oder Adjazenzmatrix) ist besser geeignet? Begründen Sie Ihre Wahl.
- Pfadsuche:** Bauen Sie den Graphen auf und nutzen Sie `bfs()` und `path()` aus `vorlesung/L08_graphen/graph.py`, um den kürzesten Pfad von S nach E zu finden. Geben Sie den Pfad und seine Länge aus.
- AoC:** Melden Sie sich bei „Advent of Code“ an und laden Sie Ihre eigene Karte herunter. Ermitteln Sie den kürzesten Pfad für die echte Karte. Messen Sie dabei die Laufzeit der BFS auf dem Beispielgelände und auf der echten Karte (z. B. mit `time.perf_counter()`). Vergleichen Sie die gemessenen Laufzeiten mit dem Verhältnis der jeweiligen $|V| + |E|$ -Werte beider Karten — bestätigt die Messung das lineare Wachstum $O(|V| + |E|)$?
- (Optional)** Nach dem Lösen der ersten Teilaufgabe gibt es eine zweite Teilaufgabe: Finden Sie den kürzesten Pfad von einem beliebigen Feld mit Höhe 'a' (oder S) zum Ziel E. Wie können Sie dieses Problem auf eine einzelne BFS zurückführen?

Aufgabe 2: Tiefensuche und topologisches Sortieren

Sie entwickeln ein Build-System für ein Softwareprojekt. Ein Modul darf erst übersetzt werden, wenn alle seine Abhängigkeiten bereits vorliegen. Die Abhängigkeiten zwischen den Modulen lauten:

Modul	hängt ab von
main	parser, codegen
parser	lexer, ast
codegen	ast, optimizer
optimizer	utils
ast	utils
lexer	(keine)
utils	(keine)

- Graph aufbauen:** Modellieren Sie die Abhängigkeiten als gerichteten azyklischen Graphen. Eine Kante von A nach B bedeutet: A hängt von B ab, B muss also vor A gebaut werden. Bauen Sie den Graphen mit `AdjacencyListGraph` auf.
- DFS ausführen:** Rufen Sie `dfs()` auf und geben Sie für jeden Knoten die Zeitmarken (aus `enter_map` und `leave_map`) aus.
- Topologische Sortierung:** Sortieren Sie die Knoten nach ihrer End-Zeitmarke in absteigender Reihenfolge. Das Ergebnis ist eine gültige Build-Reihenfolge. Überprüfen Sie, ob alle Abhängigkeiten eingehalten werden.
- Komplexität:** Die Tiefensuche hat die Zeitkomplexität $O(|V| + |E|)$. Wie viele Knoten und Kanten hat Ihr Graph? Erklären Sie anhand des konkreten Graphen, warum jeder Knoten und jede Kante genau einmal betrachtet wird.

Musterlösung Praktikum 7: Suche in Graphen

1 Breitensuche auf einer Höhenkarte

Die vollständige Lösung befindet sich im Repository unter `praktika/07_graph/aufgabe1_bfs.py`.

a) Modellierung

Jedes Feld (r, c) der Karte wird zu einem Knoten; eine gerichtete Kante existiert von (r, c) nach (r', c') , wenn (r', c') ein horizontaler oder vertikaler Nachbar ist und $\text{height}(r', c') - \text{height}(r, c) \leq 1$.

Darstellung: Eine Adjazenzliste ist deutlich besser geeignet. Das Grid ist ein lichter Graph (*sparse*): Jeder Knoten hat *maximal* vier Nachbarn, also $|E| \leq 4|V|$. Die Adjazenzmatrix hätte $|V|^2$ Einträge; für ein 41×122 -Grid wären das über 25 Millionen Einträge, obwohl nur etwa $4|V|$ davon belegt wären.

Für das 8×5 -Beispielgelände gilt:

$$|V| = 40, \quad |E| \leq 2 \cdot (7 \cdot 5 + 4 \cdot 8) = 134$$

Der Faktor 2 ergibt sich daraus, dass der Graph gerichtet ist: jedes Nachbarpaar kann bis zu zwei Kanten liefern (hin und zurück). $7 \cdot 5$ zählt die horizontalen Nachbarpaare $((C-1) \cdot R = 7 \cdot 5)$ und $4 \cdot 8$ die vertikalen $((R-1) \cdot C = 4 \cdot 8)$. Randknoten haben weniger als vier Nachbarn; außerdem sind wegen der Höhenbeschränkung nicht alle Richtungen begehbar, daher $\leq$.

b) Pfadsuche

```
g, rows, cols = build_graph(GRID)
start, end = find_start_end(GRID)
dist_map, pred_map = g.bfs(start)
pfad = g.path(end, pred_map)
print(len(pfad) - 1, "Schritte")
```

Ausgabe:

Pfadlänge: 31 Schritte

```
Pfad: (0,0) -> (1,0) -> (1,1) -> (2,1) -> (3,1) -> (3,2) -> (4,2)
      -> (4,3) -> (4,4) -> (4,5) -> (4,6) -> (4,7) -> (3,7) -> (2,7)
      -> (1,7) -> (0,7) -> (0,6) -> (0,5) -> (0,4) -> (0,3) -> (1,3)
      -> (2,3) -> (3,3) -> (3,4) -> (3,5) -> (3,6) -> (2,6) -> (1,6)
      -> (1,5) -> (1,4) -> (2,4) -> (2,5)
```

c) AoC – Zeitmessung und Vergleich

```
import time
t0 = time.perf_counter()
dist_map, pred_map = g.bfs(start)
t1 = time.perf_counter()
print(f"Laufzeit: {(t1-t0)*1000:.4f} ms")
```

Für das Beispielgelände beträgt die Laufzeit weniger als **0,1 ms**. Eine typische AoC-Puzzleeingabe hat ein $41 \times 122 = 5002$ -Knoten-Grid, also etwa $125 \times$ mehr Knoten. Da BFS linear in $O(|V| + |E|)$ wächst, sollte die Laufzeit auf der echten Karte ebenfalls um den Faktor ≈ 125 größer sein. Da absolute Zeiten auf moderner Hardware im Submillisekunden-Bereich liegen, ist die Proportionalität leichter über viele Wiederholungen (`timeit`) zu bestätigen als über eine Einzelmessung.

d) (Optional) Bester Startpunkt – AoC Teil 2

Statt BFS von jedem 'a'-Feld einzeln zu starten (was k BFS-Läufe erfordert, wobei k die Anzahl der 'a'-Felder ist), kann man den Graphen *umkehren* (alle Kanten drehen) und eine einzige BFS von E starten. Der erste erreichbare 'a'-Knoten (d. h. der mit der kleinsten Distanz zu E im umgekehrten Graphen) liefert den kürzesten Gesamtpfad. Die Gesamtkomplexität bleibt $O(|V| + |E|)$.

2 Tiefensuche und topologisches Sortieren

Die vollständige Lösung befindet sich im Repository unter `praktika/07_graph/aufgabe2_dfs.py`.

a) Graph aufbauen

Kantrichtung: Eine Kante verläuft von `dep` nach `module`, wenn `module` von `dep` abhängt – also in der Richtung „*muss vorher gebaut werden*“. Damit entspricht die Kantrichtung genau der Voraussetzungsbeziehung, die das topologische Sortieren auflösen soll.

```
for module, deps in DEPENDENCIES.items():
    for dep in deps:
        g.connect(dep, module)    # dep muss vor module gebaut werden

g.graph("BuildDependencies")
```

Der Graph hat $|V| = 7$ Knoten und $|E| = 8$ Kanten.

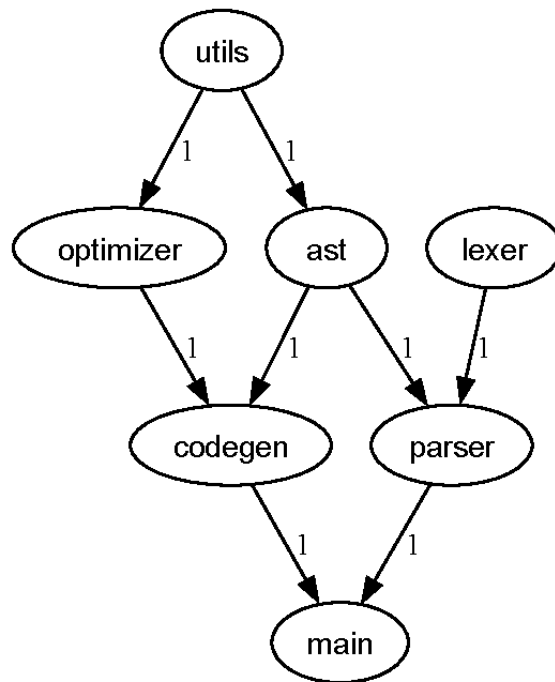


Abbildung 1: Abhängigkeitsgraph des Build-Systems (erzeugt mit Graphviz)

b) DFS – Zeitmarken

```
enter_map, leave_map, pred_map = g.dfs()
```

Modul	begin	end
parser	1	4
main	2	3
codegen	5	6
optimizer	7	8
ast	9	10
lexer	11	12
utils	13	14

Die genauen Zeitmarken hängen von der Iterationsreihenfolge des Sets `white_vertices` ab und können sich zwischen Python-Versionen unterscheiden. Entscheidend ist die relative Ordnung der End-Zeitmarken.

c) Topologische Sortierung

Knoten nach End-Zeitmarke absteigend sortiert:

`utils` → `lexer` → `ast` → `optimizer` → `codegen` → `parser` → `main`

Überprüfung: Für jede Abhängigkeit muss die Voraussetzung *vor* dem abhängigen Modul in der Reihenfolge erscheinen.

Abhängigkeit	Voraussetzung vor Modul?	
main $\leftarrow$ parser	parser vor main	✓
main $\leftarrow$ codegen	codegen vor main	✓
parser $\leftarrow$ lexer	lexer vor parser	✓
parser $\leftarrow$ ast	ast vor parser	✓
codegen $\leftarrow$ ast	ast vor codegen	✓
codegen $\leftarrow$ optimizer	optimizer vor codegen	✓
optimizer $\leftarrow$ utils	utils vor optimizer	✓
ast $\leftarrow$ utils	utils vor ast	✓

Alle Abhängigkeiten sind eingehalten.

d) Komplexität

Die Tiefensuche hat die Zeitkomplexität $O(|V| + |E|)$; im konkreten Graphen gilt $|V| = 7$ und $|E| = 8$.

Warum wird jeder Knoten genau einmal betrachtet? Jeder Knoten durchläuft genau einmal den Übergang weiß $\rightarrow$ grau (Zeitpunkt *begin*) und einmal grau $\rightarrow$ schwarz (Zeitpunkt *end*). Die Farbprüfung `color == WHITE` in `dfs_visit` stellt sicher, dass ein bereits besuchter Knoten nicht erneut besucht wird. Im Beispiel: 7 Knoten $\times$ 2 Zeitmarkenereignisse = 14 Operationen $\Rightarrow O(|V|)$.

Warum wird jede Kante genau einmal betrachtet? Beim Besuch eines Knotens v (d.h. wenn v grau wird) iteriert `dfs_visit` über alle ausgehenden Kanten $v \rightarrow u$. Da jeder Knoten genau einmal besucht wird, wird auch jede seiner ausgehenden Kanten genau einmal untersucht. Im Beispiel: 8 Kanten $\Rightarrow O(|E|)$.

Insgesamt folgt: $O(|V| + |E|)$.